

Scelte Progettuali

Progetto 2: console.log (Django)

Programmazione Web — A.A. 2025/2026

Gruppo di Lavoro:

Riccardo Nasatti – Matricola 1092857

Tommaso Rota – Matricola 1092926

Tecnologie Utilizzate:

Python 3, Django 4.2, Bootstrap 5, SQLite

Indice

1	Migrazione da PHP a Django	2
2	Database: da MySQL a SQLite	2
3	Architettura Django	2
3.1	Pattern architetturali usati	2
4	Interfaccia Utente	2
4.1	Filtri avanzati	3
5	Paginazione	3
6	Gestione Utente Attivo	3
7	Regole di Business Implementate	3
8	Appendice: Correzioni post Progetto 1	4

1 Migrazione da PHP a Django

Il progetto 1 era scritto in PHP con MySQL. Per il progetto 2 abbiamo effettuato il refactoring completo verso Python e il framework Django, mantenendo la stessa struttura del database e la stessa logica applicativa.

Motivazione: Django fornisce un sistema di template, routing, protezione CSRF e ORM integrati, rendendo il codice molto più manutenibile e sicuro rispetto al PHP procedurale del progetto 1.

2 Database: da MySQL a SQLite

Il progetto 1 richiedeva un server MySQL esterno (XAMPP) per funzionare. Nel progetto 2 abbiamo migrato a **SQLite**, che è integrato in Python e non richiede installazioni aggiuntive.

Vantaggi:

- **Zero configurazione:** il database viene creato automaticamente al primo avvio
- Il progetto rispetta la **regola dei 5 minuti**: basta un doppio clic su `avvia.bat`
- **Portabilità:** il database è un singolo file (`db.sqlite3`) che può essere copiato e condiviso

Schema: Lo schema del database è identico a quello del progetto 1 (tabelle `Utente`, `Quiz`, `Domanda`, `Risposta`, `Partecipazione`, `RispostaUtenteQuiz`), tradotto in modelli Django (`models.py`).

Importazione dati: Il comando `python manage.py import_legacy_db` legge il file SQL del progetto 1 e popola il database SQLite. Questo processo è automatizzato in `avvia.bat`.

3 Architettura Django

<code>consolelog_django/</code>	<code><- Configurazione del progetto (settings.py, urls.py)</code>
<code>quiz/</code>	<code><- App principale</code>
<code>models.py</code>	<code><- Modelli del database (6 tabelle)</code>
<code>views.py</code>	<code><- Logica applicativa</code>
<code>urls.py</code>	<code><- Routing URL</code>
<code>templates/quiz/</code>	<code><- Template HTML</code>
<code>static/quiz/</code>	<code><- CSS e JavaScript</code>
<code>management/</code>	<code><- Comandi Django custom (import_legacy_db)</code>

3.1 Pattern architetturali usati

- **MTV (Model-Template-View):** il pattern Django equivalente all'MVC tradizionale
- **Query SQL raw:** mantenute per rispettare la logica originale e per query complesse con JOIN multipli
- **AJAX:** le tabelle filtrabili usano chiamate `$.ajax()` jQuery per aggiornare solo il corpo tabella senza ricaricare la pagina

4 Interfaccia Utente

- **Framework CSS:** Integrazione di **Bootstrap 5** (per i componenti standard e responsività) combinata con Vanilla CSS personalizzato (`style.css`) per preservare il design originale garantendo al contempo pulizia e manutenibilità.

- **Font:** Inter (Google Fonts) — font con spaziatura mono-spaziale per i numeri, garantendo l'allineamento delle colonne numeriche
- **Palette colori:** Giallo (#F5C518) su sfondo scuro (#1A1A1A), con accent colors per successo/errore/info
- **Layout:** Header fisso + barra navigazione + area principale con sidebar filtri e contenuto centrale

4.1 Filtri avanzati

- **Ricerca con debounce:** 350ms di ritardo prima di inviare la query AJAX, per evitare troppe richieste al server durante la digitazione
- **Filtro stato quiz:** Checkbox multiple (al posto del menu a tendina) per poter selezionare contemporaneamente gli stati (Aperto / Chiuso / Futuro)
- **Filtri utente avanzati (Datalist intelligente):** Campo testo con datalist per l'autocompletamento. Per evitare di caricare liste lunghissime nel DOM, la lista delle opzioni (es. per Utente o Creatore) si aggancia all'input solo dopo aver digitato almeno **3 caratteri**.

5 Paginazione

Tutte le tabelle sono paginate lato server usando il Paginator di Django:

- Tabella quiz
- Tabella utenti
- Tabella partecipazioni

L'ordinamento avviene lato server con una whitelist di colonne ammesse (per prevenire SQL injection).

6 Gestione Utente Attivo

Poiché il progetto non ha un sistema di autenticazione, l'utente "attivo" viene gestito con `sessionStorage` del browser (visibile solo nella tab corrente). L'utente può selezionarsi dal selettore in alto a destra nell'header.

Questa funzionalità serve principalmente per partecipare ai quiz e per identificare i propri quiz nella lista (mostrando i pulsanti Modifica/Elimina solo per i quiz dell'utente attivo).

7 Regole di Business Implementate

- **Modifica quiz:** non consentita se il quiz ha già partecipazioni registrate
- **Eliminazione quiz:** non consentita se il quiz ha già partecipazioni registrate
- **Creazione quiz senza domande:** consentita — le domande possono essere aggiunte successivamente
- **Partecipazione:** possibile solo nei quiz aperti (`dataInizio ≤ oggi ≤ dataFine`)
- **Calcolo punteggio:** somma dei punti delle risposte corrette selezionate; se si sceglie anche una risposta sbagliata per una domanda, la domanda vale 0; se si scelgono tutte le risposte per una domanda, la domanda vale 0.

8 Appendice: Correzioni post Progetto 1

Sulla base del feedback del docente, abbiamo apportato le seguenti correzioni:

Problema segnalato	Soluzione adottata
Stili inline nei tag HTML	Completamente rimossi dai template e spostati in <code>style.css</code> con classi dedicate e posizionamenti gestiti correttamente.
JavaScript inline nei script	Spostato in un unico file <code>main.js</code> , con variabili di stato e logiche asincrone centralizzate (sfruttando il cache buster per aggiornamenti immediati).
Alert del browser per errori	Sostituiti con <code>div class="alert"</code> integrati nella UI (inline nel form).
Paginazione assente	Aggiunta paginazione server-side intelligente con Paginator Django su tutte le tabelle.
Ordinamento: nessun indicatore	Aggiunta freccia su/giù nell'intestazione colonna ordinata con logica lato server sicura.
Filtro stato quiz	Sostituito con checkbox multiple per selezionare contemporaneamente gli stati desiderati, con layout flex affiancato al badge.
Filtro creatori (menu a tendina)	Sostituito con input text collegato a un Datalist che si attiva intelligentemente solo digitando almeno 3 caratteri.
Filtri utente (HAVING)	Risolto il problema che impediva di filtrare correttamente per numero di quiz/partecipazioni (usando query <code>HAVING</code> corrette e <code>COUNT(DISTINCT)</code>).
Contatore risultati assente	Aggiunto totale trovati (es. "X quiz trovati") su tutte le tabelle, che si aggiorna in tempo reale.
Modifica/Eliminazione quiz	I bottoni per la modifica e l'eliminazione sono nascosti dinamicamente se il quiz non soddisfa i requisiti, bloccando inoltre l'azione via backend.
Bug Modifica form	Risolto crash differenziando le richieste di caricamento del form dalla reale sottoposizione POST.
Etichetta errata ("Annullata")	Corretta in "Risposta errata" / "Sbagliata".
Filtro partecipazioni (utente, %)	Corretto: il filtro usa la percentuale coerentemente, e l'accesso diretto da click sui numeri in tabella popola e applica automaticamente il filtro per utente.
MySQL richiesto (XAMPP)	Migrazione totale a SQLite (<code>db.sqlite3</code>) — nessun server esterno necessario, rendendo l'app immediatamente utilizzabile (zero config).